

Settings-Group / Config-File Strategy (Proposal)

Status: design + working examples only. Nothing described here is wired into `asl-win11maker.ps1` or `asl-win11.functions.ps1` yet — the example files under `config/examples/` are real, loadable PowerShell data files you can dot-source and inspect today, but the actual pipeline still reads `$appPackagePrefixes / $scheduledTaskPaths` /the hand-written `Disable-*` call list exactly as it always has. Wiring this in is future work, flagged explicitly at the end of this doc.

Problem this solves

Today's customization surface (see `CLAUDE.md`'s "Three arrays near the top of the script" and the `OPTIONAL` /comment-out convention) is good for *one axis at a time*: want more aggressive package removal? Uncomment some lines. Want to keep corporate apps? Pass `-KeepCorporateApps`, which is a single flat boolean carve-out. Want a different hardware-compatibility posture? Pass `-BypassMode`. These don't compose cleanly — e.g. there's no way today to ask for "aggressive package removal" *and* "keep Teams" *and* "bypass + inject drivers" without manually re-deriving which lines to comment/uncomment each time, and every such combination has to be remembered and re-applied by hand on the next build.

`docs/build-options-reference.md` already identified the natural axes from what the code *actually* branches on: package aggressiveness, account mode, and hardware compatibility are independent concerns that each want their own named settings group, selectable and combinable.

Proposed model: three independent axes, each a small `.ps1` data file

No JSON/YAML — per the user's ask, plain PowerShell files you dot-source, each returning a hashtable or array literal. Every item is `$true / $false` -toggleable or comment-out-able, same spirit as today's arrays, just reorganized so *groups* of toggles have names.

```

config/
  examples/
    categories/                                # one file per tweak category: every known item, tagged
      packages.categories.ps1
      scheduled-tasks.categories.ps1
      registry-tweaks.categories.ps1
      hardware-compat.categories.ps1
      account-mode.categories.ps1
    profiles/                                # one file per named group: filters/selects from a
categories file
  packages.minimal.ps1
  packages.reduced.ps1
  packages.normal.ps1
  packages.corporate.ps1
  scheduled-tasks.minimal.ps1
  scheduled-tasks.default.ps1
  registry-tweaks.minimal.ps1
  registry-tweaks.default.ps1
  account-mode.local.ps1
  account-mode.online.ps1
  hardware-compat.strict-compliant.ps1
  hardware-compat.bypass-offline.ps1
  hardware-compat.bypass-and-drivers.ps1

```

Categories files: tag every known item once

Each item gets a numeric **Tier** (how aggressively it's removed/changed — lower tier = removed/changed by more profiles, since it's less controversial) plus a **CorporateExempt** flag where relevant, plus a **Source** pointer back into `docs/tweak-catalog.md` / `docs/build-options-reference.md` so provenance never gets lost. This is a **best-guess starting classification** — see each file's header comment for the judgment calls made and adjust freely; nothing about the tier numbers is load-bearing outside these example files.

Concretely, for packages (`config/examples/categories/packages.categories.ps1`):

- **Tier 1** — unambiguous OEM/Xbox/gaming-adjacent bloat nobody building a trimmed image wants (Dolby, Intel Management, Xbox family, Solitaire, 3D Viewer, Wallet, Zune Music/Video, Clipchamp, Mixed Reality, Skype, Cortana, Family Safety, Power Automate Desktop, Office push notifications). Removed starting at the **Normal** profile — i.e. even the lightest-touch profile removes these.
- **Tier 2** — today's remaining active-by-default items: broadly useful to *some* users but removed by this repo's existing default (Mail & Calendar, OneNote, People, Maps, Camera, Sound Recorder, Sticky Notes, To Do, Feedback Hub, Quick Assist, Dev Home, Start recommendations, Copilot, Teams, Outlook, Phone Link, Get Help/Started, Cross Device). Removed starting at **Reduced** (this matches today's actual default behavior — "Reduced" is the new name for what the script already does out of the box).
- **Tier 3** — today's commented-out, more aggressive/niche removals (AI/Recall-era packages, Photos, Widgets board, MPEG-2/web media codecs, Screen Sketch, Store purchase helper). Removed only at **Minimal**.

- **Tier 0 ("keep always")** — `Microsoft.Paint`, `Microsoft.MSPaint`, `Microsoft.WindowsTerminal`: already commented out upstream for being broadly-useful mainstream apps, not bloat. No profile removes these; they're not even in the tiered set, by design.
- `CorporateExempt = $true` on the 6 items today's `-KeepCorporateApps` carves out (Copilot ×2, Teams ×3, Outlook). This is an **overlay**, independent of tier — the **Corporate** profile is "Reduced tier, with the exempt overlay applied," not a tier of its own. Document this clearly rather than pretending Corporate sits on the same Minimal→Normal ladder; it doesn't.

Scheduled tasks and registry tweaks follow the same two-tier shape (today's 5 active tasks / 11 active registry functions = Tier 1; today's commented-out 6 WU-task paths / 3 registry functions = Tier 2 → Minimal only). See each categories file's header for the exact list.

Hardware-compatibility and account-mode don't have a removal "tier" — they're small, named, mutually exclusive strata mapping directly onto `-BypassMode` × driver switches, and `-LocalAccountName` vs. default/ `-KeepCorporateApps`, respectively. See those two categories files directly; they're short enough not to need a tiering scheme.

Profile files: pick a tier threshold (or a named stratum) and filter

A profile file is intentionally tiny — it dot-sources the categories file and filters:

```
# profiles/packages.reduced.ps1 — matches this repo's current out-of-the-box default.
$categories = . (Join-Path $PSScriptRoot '..\categories\packages.categories.ps1')
return $categories | Where-Object { $_.Tier -le 2 }
```

Combining axes is then just picking one profile file per axis and merging their outputs — no interaction effects to hand-manage, because each axis only ever touches its own category's array.

Intended (future, not built now) loading mechanism

A new `-SettingsProfile` parameter set on `asl-win11maker.ps1` — three parameters, one per axis (e.g. `-PackageProfile <path>`, `-AccountModeProfile <path>`, `-HardwareCompatProfile <path>`), each defaulting to the file matching today's actual behavior (`packages.reduced.ps1`, `account-mode.online.ps1` unless `-LocalAccountName` is given, `hardware-compat.strict-compliant.ps1` matching today's `-BypassMode None` default) so an un-opinionated invocation behaves identically to today. Each resolves to an array/hashtable dot-sourced early in `Initialize-Tiny11Session`, replacing the current hand-written `$appPackagePrefixes` / `$scheduledTaskPaths` literals and the `if ($KeepCorporateApps) / if ($Core)` branches scattered through the registry-tweak section of the pipeline — those branches become "is this tweak's tag present in the resolved profile," a single `Where-Object`-driven loop instead of one `if` per tweak.

This is a real refactor of `asl-win11maker.ps1`'s customization surface and deserves its own review pass once the taxonomy above has been used for a few real builds and any tier misjudgments have shaken out — explicitly **not** attempted in this documentation pass.

See also

- `docs/build-options-reference.md` — where the three axes (package aggressiveness, account mode, hardware compatibility) were identified from the actual CLI surface.
- `docs/optimization-checklists.md` — the human-facing decision checklists this config system is meant to eventually encode as data instead of a checklist you re-read every time.